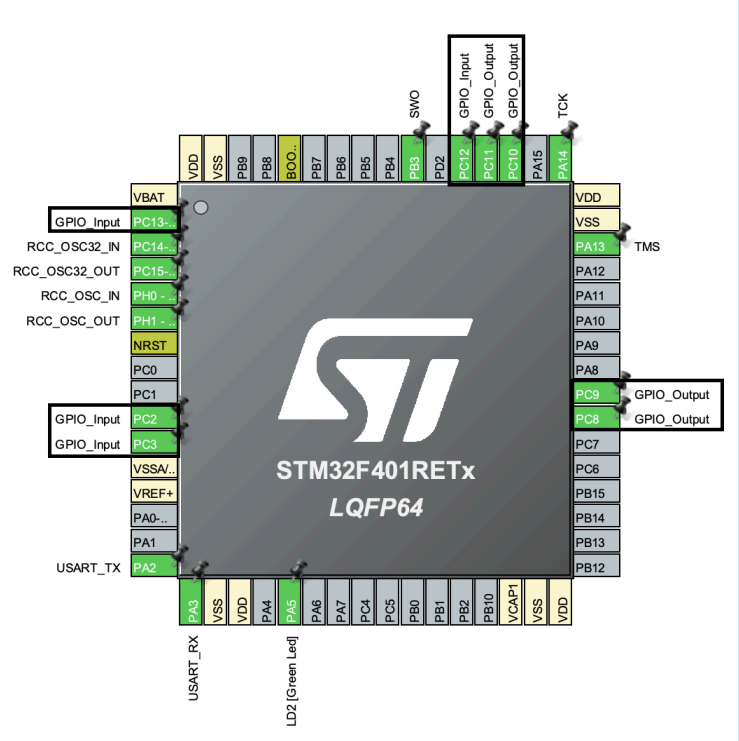
Mark	10/11

Team name:	B9		
Homework number:	Homework 10		
Due date:	25/11/2025		
Contribution:	No	Partial	Full
Pica Mirko			✅
Prendin Christian			✅
Pignataro Gabriele			✅
Moazenkakhki Mostafa			✅
Yarrabbotula Sunandha			✅
Notes:			

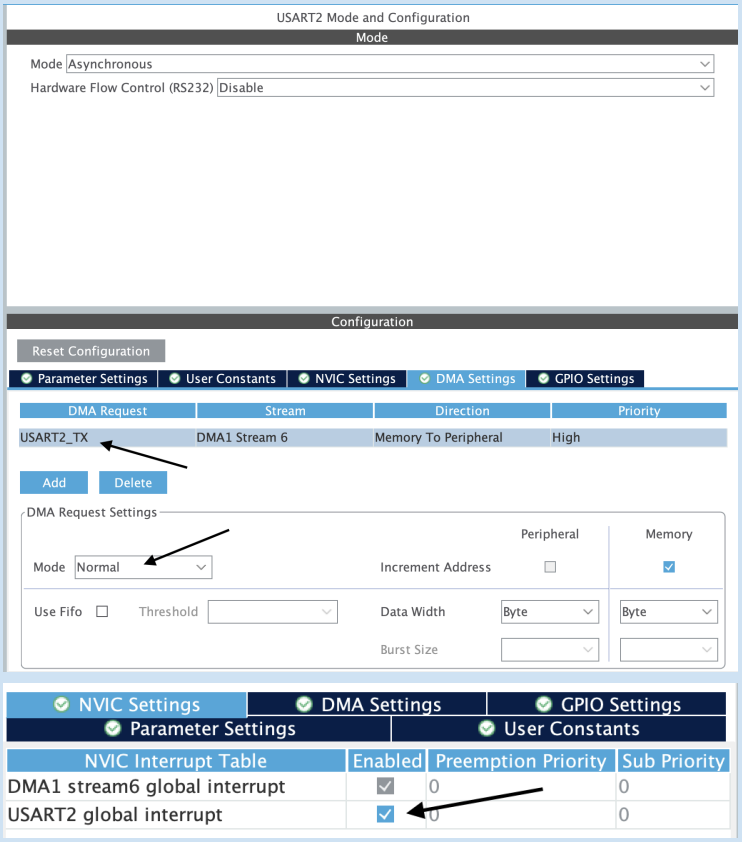
Project name	Keyboard & Encoder		
Not done	Partially done (major problems)	Partially done (minor problems)	Completed
			✅

Part 1. Finish the Keyboard project.

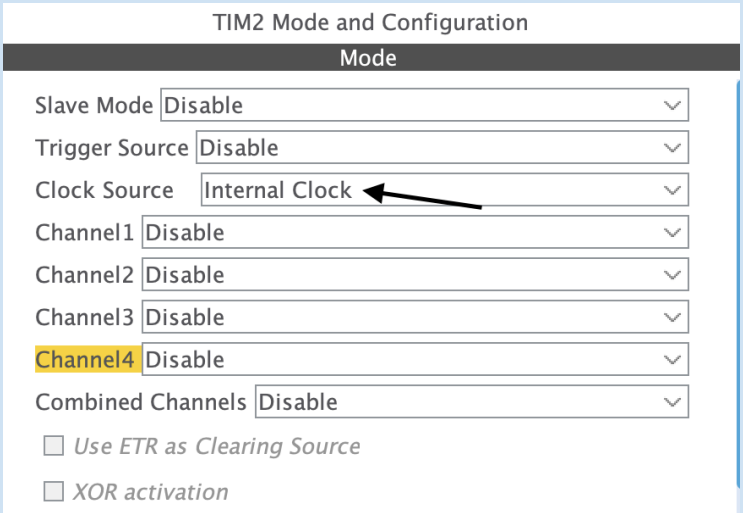
The first step is to set up the correct pins to communicate with the keyboard inside the ".ioc" file (as indicated in the slide).

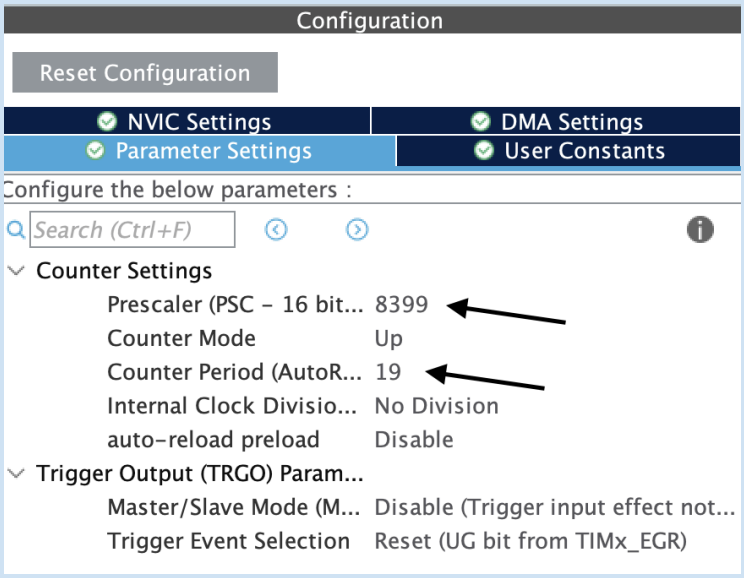


In the connectivity section we enable the USART2 and we enable the interrupt and the DMA in transmission (as done in previous homeworks) to send the keyboard value to the remote terminal.



Now to trigger the reading every 2 millisecond we have to configure TIM2 and its global interrupt. We set the clock source, the prescaler at 8399, the counter at 19 and enable the global interrupt.





We go into the “main.c” file. In the main function we enable the first column and start the timer.

```
  /* USER CODE BEGIN 2 */
  HAL_GPIO_WritePin(GPIOC, COLUMN_PINS[current_column], GPIO_PIN_SET);
  HAL_TIM_Base_Start_IT(&htim2);
  /* USER CODE END 2 */
```

Now we just need to write the timer callback. If a key is pressed we increment the counter, otherwise we reset its counter. When a key is pressed for enough time (20ms) send it to the remote terminal (the check for <255 is to prevent overflow, we don’t care if the key is still pressed anyway). At the end of the callback we switch to the next column.

```
/* Private user code ---------------------------------------------------------*/
/* USER CODE BEGIN 0 */
void sendToTerminal(char c) {
    int length = snprintf(string, sizeof(string), "%c", c);
    HAL_UART_Transmit_DMA(&huart2, (uint8_t *)string, length);
}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim->Instance == TIM2) {
        for (int i = 0; i < 4; i ++) {
            if (!HAL_GPIO_ReadPin(GPIOC, ROW_PINS[i])) {
                if (key_counter[i][current_column] < 255) {
                    key_counter[i][current_column]++;
                    if(key_counter[i][current_column] == 10)
                        sendToTerminal(KEY_MAP[i][current_column]);
                }
            } else {
                key_counter[i][current_column] = 0;
            }
        }

        HAL_GPIO_WritePin(GPIOC, COLUMN_PINS[current_column], GPIO_PIN_RESET);
        current_column = (current_column + 1) % 4;
        HAL_GPIO_WritePin(GPIOC, COLUMN_PINS[current_column], GPIO_PIN_SET);
    }
}
/* USER CODE END 0 */
```

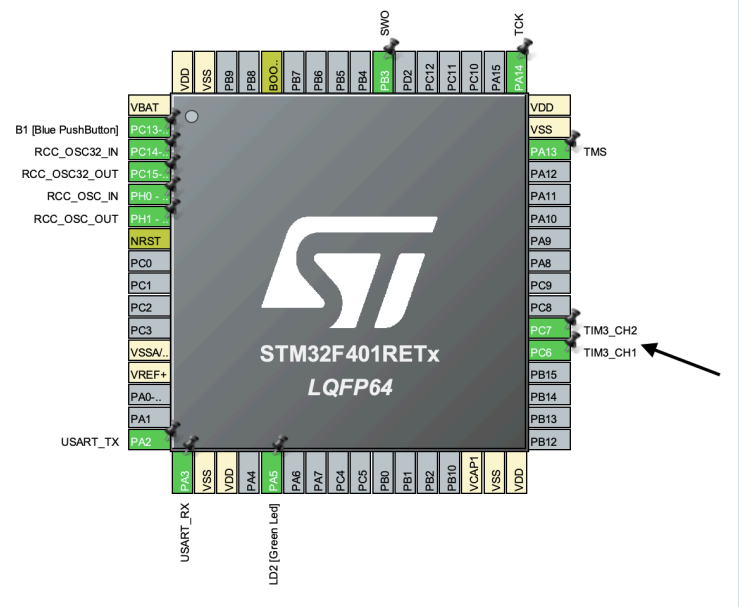
To get it working we need to include the “stdio.h” library for the snprintf and define all the variables used.

```
/* USER CODE BEGIN PV */
const uint16_t ROW_PINS[4] = {GPIO_PIN_3, GPIO_PIN_2, GPIO_PIN_13, GPIO_PIN_12};
const uint16_t COLUMN_PINS[4] = {GPIO_PIN_11, GPIO_PIN_10, GPIO_PIN_9, GPIO_PIN_8};

const char KEY_MAP[4][4] = {
        {'F', 'E', 'D', 'C'},
        {'B', 'A', '9', '8'},
        {'7', '6', '5', '4'},
        {'3', '2', '1', '0'}};

volatile int current_column = 0;
volatile uint8_t key_counter[4][4] = {0};

char string[8];
/* USER CODE END PV */
```

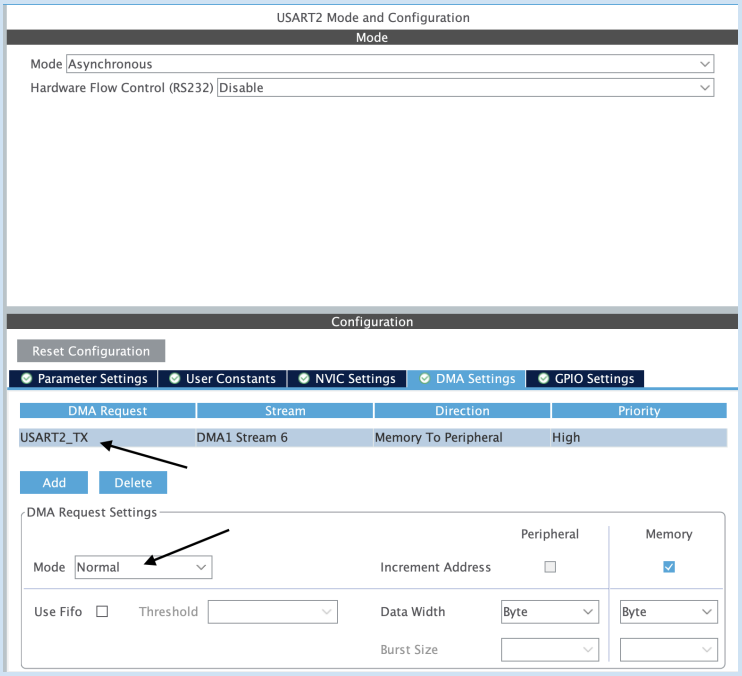
We tested it and it seems to be working fine.

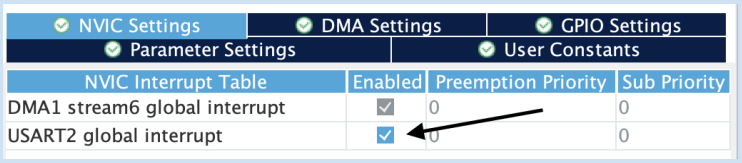
Part 2. Finish the Encoder project.

The first step is always to understand what pins we need to use and enable them in the “.ioc” file (this time it was indicated in the slides).

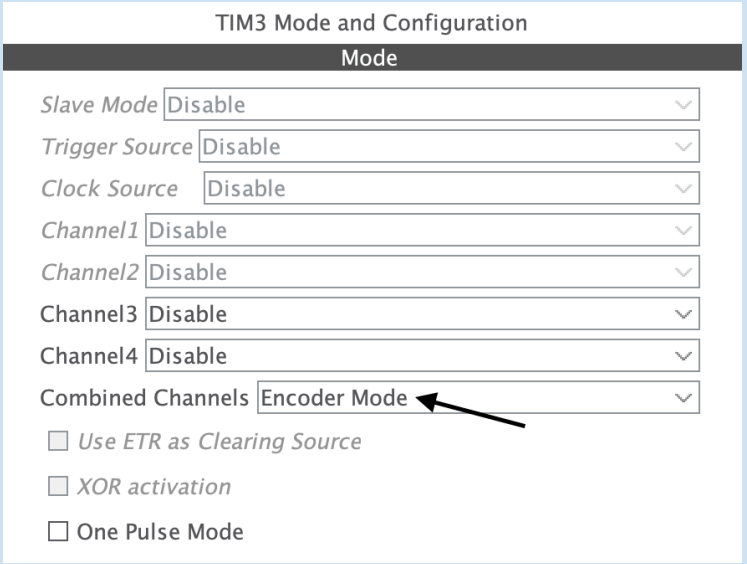


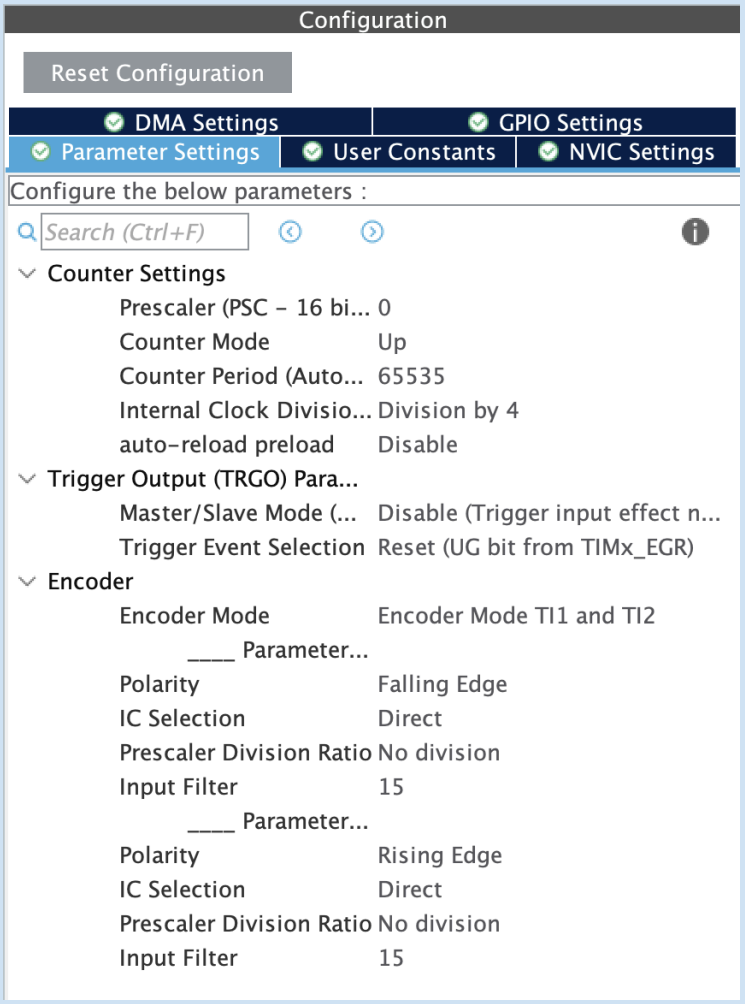
In the connectivity section we enable the USART2 and we enable the interrupt and the DMA in transmission (as done in previous homeworks) to send the encoder value to the remote terminal.



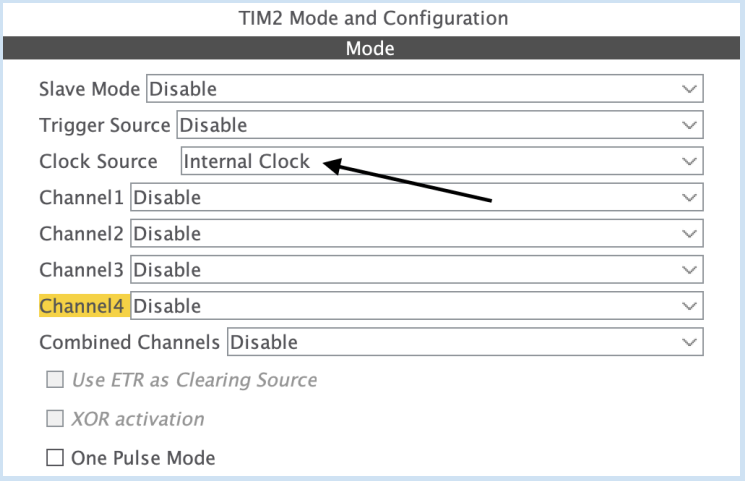


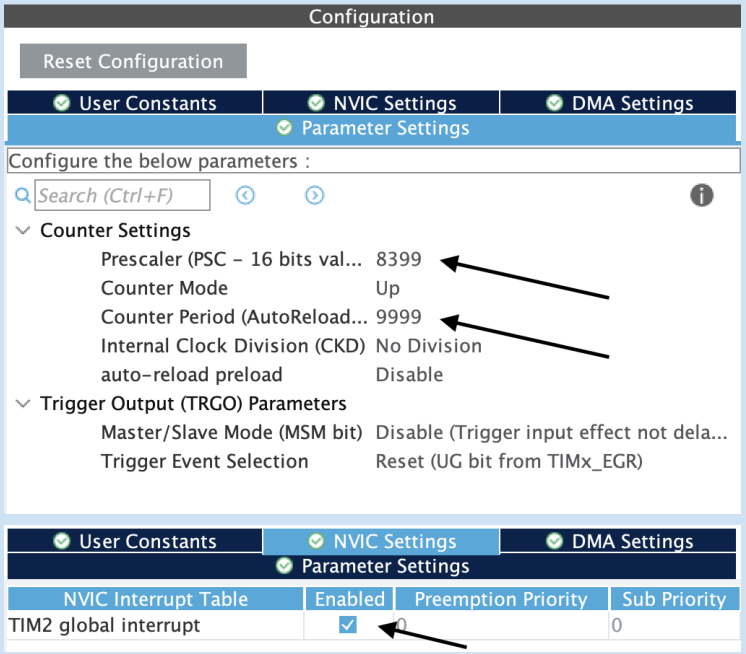
We set up TIM3 to operate in encoder mode, with the correct settings and filter (shown in class).





Now to trigger the reading every second we have to configure TIM2 and its global interrupt. We set the clock source, the prescaler at 8399, the counter at 9999 and enable the global interrupt.





We go into the “main.c” file. In the main function we start TIM3 in encoder mode and start TIM2 to provide an interrupt every second.

```
/* USER CODE BEGIN 2 */
HAL_TIM_Encoder_Start(&htim3, TIM_CHANNEL_ALL);
HAL_TIM_Base_Start_IT(&htim2);
/* USER CODE END 2 */
```

Now we just need to write the timer callback. We first get the value of the encoder, compute the delta and convert it to rpm. We just update the old encoder value and send the rpm to the remote terminal. The encoder values are unsigned 16 bit integers but when we do the difference we cast them to an 16 bit integer to fix every underflow or overflow problems.

```
/* Private user code ---------------------------------------------------------*/
/* USER CODE BEGIN 0 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim->Instance == TIM2) {
        uint16_t encoder = __HAL_TIM_GET_COUNTER(&htim3);
        int16_t delta = (int16_t) (encoder - encoder_old);
        float rpm = (float) delta * 60 / 48;
        encoder_old = encoder;
        int length = snprintf(string, sizeof(string), "RPM: %.2f\n", rpm);
        HAL_UART_Transmit_DMA(&huart2, (uint8_t *)string, length);
    }
}
/* USER CODE END 0 */
```

To get it working we need to include the “stdio.h” library for the snprintf, enable the float in the

printf and define all the variables used.

```
/* USER CODE BEGIN PV */
volatile uint16_t encoder_old = 0;

char string[32];
/* USER CODE END PV */
```

We tested it and it seems to be working fine.

Professor comments:	